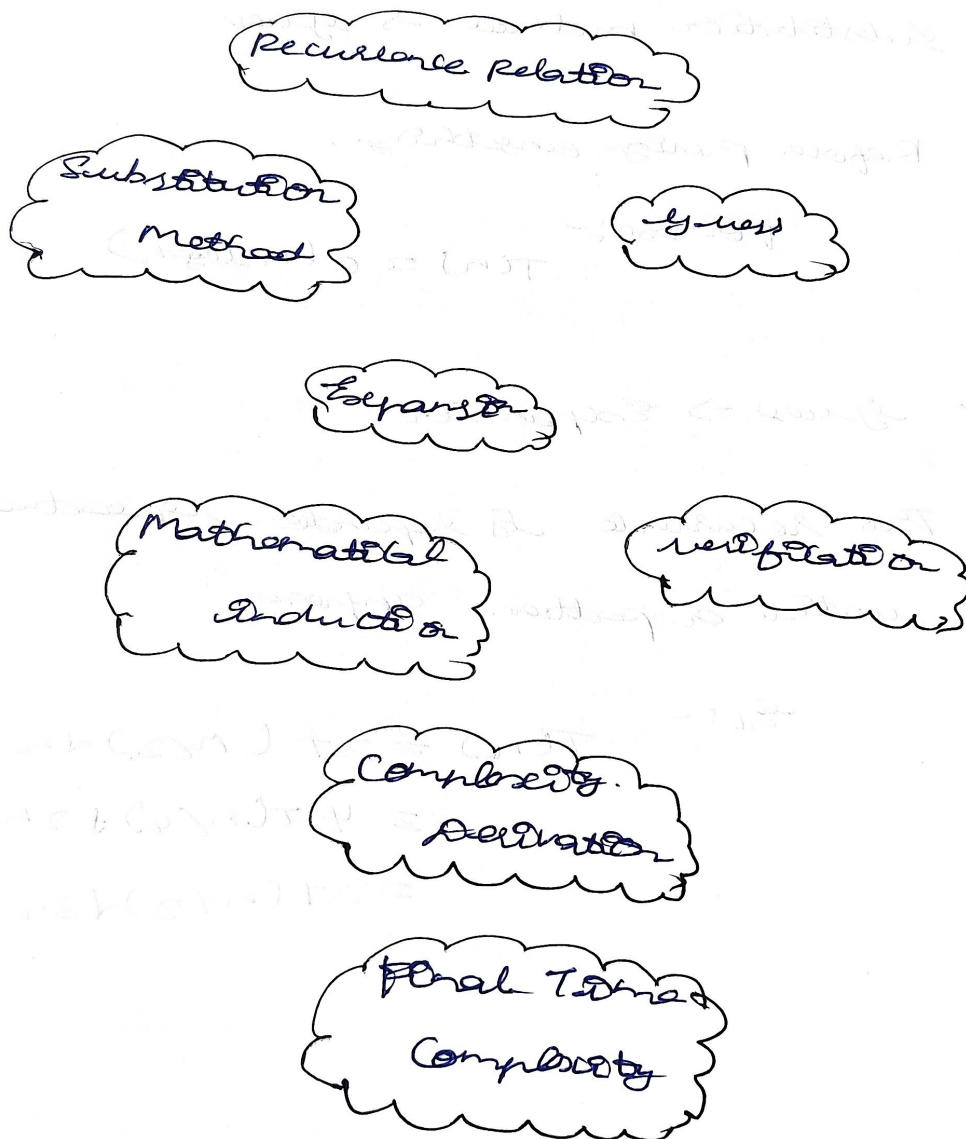


3. Tracing the path from a Recurrence Relation to its closed form complexity.

The process of solving a recurrence using the substitution method can be represented through

concept map



1. Recurrence Relation $\rightarrow$ substitution method

$$\text{Ex: - } T(n) = 2T\left(\frac{n}{2}\right) + n$$

Purpose: Converts the recursive equation into a solvable mathematical form

2. Substitution method $\rightarrow$ guess

Before proving anything,

For ex:-

$$T(n) = O(n \log n)$$

3. Guess $\rightarrow$ Expansion

The recurrence is expanded repeatedly until a pattern appears.

Ex:-

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$= 4T\left(\frac{n}{4}\right) + 2n$$

$$= 8T\left(\frac{n}{8}\right) + 3n$$

5. Induction $\rightarrow$ verification

verification checks that

- the induction hypothesis is correct,
- the base is satisfied.

for ex:-

$$n \geq 1$$

$$T(1) = 1$$

6. verification $\rightarrow$ complexity derivation

once the proof is verified, we derive the final asymptotic complexity using Big-O

for ex:-

$$T(n) = O(n \log n)$$

7. complexity derivation $\rightarrow$ final time complexity

for ex:-

- $O(\log n)$ - very efficient (binary search)
- $O(n)$ - linear time.
- $O(n \log n)$ - Efficient divide & conquer algorithms.

- $O(n^2)$ - Quadratic algorithm.
- $O(2^n)$ - Exponential algorithm.

Conclusion

The extended content now illustrates how a recurrence relation is systematically transformed into its final closed-form complexity starting with the recurrence, the substitution method involves making a guess.

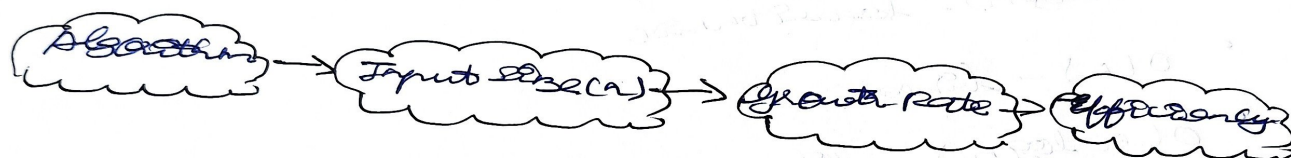
4. Concept Map: Algorithm $\rightarrow$ Input Size (n)

$\rightarrow$ Growth Rate $\rightarrow$ Efficiency.

The relationship between an algorithm and its performance can be understood through.

Concept Map

Asymptotic Notation



1. Algorithm $\rightarrow$ Input Size (n)

An algorithm is a step by step procedure for solving a problem. Its performance depends on the Input Size (n)

Ex:-

Searching 10 numbers, takes less time than searching 10,000 numbers

1, 2,

Consider:

As the input size increases, the algorithm performs more operations, affecting its execution time.

2. Input size (n) $\rightarrow$ growth rate

Examples of common growth rates

- $O(1)$ - Constant
- $O(\log n)$ - logarithmic
- $O(n)$ - linear
- $O(n \log n)$ - linearithmic
- $O(n^2)$ - Quadratic
- $O(2^n)$ - Exponential

3. Efficiency $\rightarrow$ Scalability

Scalability refers to an algorithm's ability to maintain acceptable performance.

Ex -

- Merge sort ($O(n \log n)$) scales well for large datasets.
- Bubble sort ($O(n^2)$) becomes inefficient as the dataset grows.

Overall Relationship

Algorithm

Process Input Size

Determine Growth Rate

Measured using Asymptotic

Indicates Algorithm efficiency

Determine Scalability

Interpretation of Concept map

- An algorithm states the process by defining the steps required to solve a problem.
- The input size (n) determines how much the algorithm must perform
- As n increases, the growth rate shows how quickly the required time or memory increases

Conclusion

The seconded concept map clearly illustrates the relationship between input growth and algorithm performance. As the input size (n) increases, the algorithm's growth rate determines how rapidly its resource usage increases.